Ingeniería del Software II

Tema 8

REINGENIERÍA

Conceptos fundamentales. La reingeniería de los procesos del negocio. Principios de reingeniería de procesos de Negocio. Reingeniería del Software. Mantenimiento. Un modelo de proceso de Reingeniería del software. Ingeniería Inversa. Ingeniería Inversa desde la Interfaz de usuario. Herramientas Case e Ingeniería Inversa.

Conceptos fundamentales

En la actualidad, existen compañías importantes que poseen decenas de miles de programas de computadora que prestan apoyo a las «viejas reglas del negocio». Cuando los administradores trabajan para modificar estas reglas y lograr así una mayor efectividad y competitividad, el software seguirá el mismo ritmo. En algunos casos, esto implica la creación de sistemas nuevos e importantes basados en computadora'. Pero en muchos otros, esto implica la modificación y/o reconstrucción de las aplicaciones existentes.

Características Básicas.

¿Qué es? cualquier producto de tecnología que está envejeciendo, se rompe con frecuencia, tarda en repararse y ya no representa la última tecnología. ¿Qué se puede hacer? En algunos casos, probablemente lo tirará y se comprará uno nuevo; pero en otros, no dispondrá de la opción de tirarlo. Necesitará reconstruirlo. Creará un producto con una funcionalidad nueva, un mejor rendimiento y fiabilidad, y un mantenimiento mejorado. Eso es lo que llamamos reingeniería.

Características Básicas.

Quién lo hace?

A nivel de negocio, la reingeniería es ejercida por especialistas de negocio (frecuentemente empresas de consultoría).

A nivel de software, la reingeniería es ejecutada por ingenieros del software.

Características Básicas.

¿Por qué *es importante?* El mundo sufre constantes cambios. Las demandas de funciones de negocios y de tecnología de información que las soportan cambian a un ritmo que impone mucha presión competitiva en todas las organizaciones comerciales. Tanto los negocios como el software que soportan (o es) el negocio deberán diseñarse una vez más para mantener el ritmo.

Características Básicas.

¿Cuáles son los pasos? La reingeniería de procesos de negocios WN) define las metas comerciales, identifica y evalúa los procesos de negocios existentes, y crea procesos comerciales revisados que mejoran las metas actuales. El proceso de reingeniería del software acompaña el análisis de inventarios, la estructuración de documentos. la ingeniería inversa, la reestructuración de datos y la ingeniería avanzada. El objetivo de estas actividades es crear versiones nuevas de los programas existentes que exhiben una mantenibilidad de mayor calidad.

Características Básicas.

¿Cuál es el producto obtenido? Son varios los productos que se elaboran (por ejemplo, modelos análisis, modelos de diseño, procedimientos de pruebas). El resultado final es un proceso de reingeniería de negocios y/o el software de reingeniería que lo soporta.

Características Básicas.

¿Cómo puedo esta seguro de que se he hecho correctamente? Utilizando las mismas prácticas que se aplican en todos los procesos de ingeniería del software, las revisiones técnicas formales evalúan los modelos de análisis y de diseños, las revisiones especializadas tienen en consideración la capacidad de aplicación comercial y la compatibilidad, y la comprobación se aplica para descubrir los errores en el contenido, en la funcionalidad y en la interoperabilidad.

Reingeniería de procesos de negocio

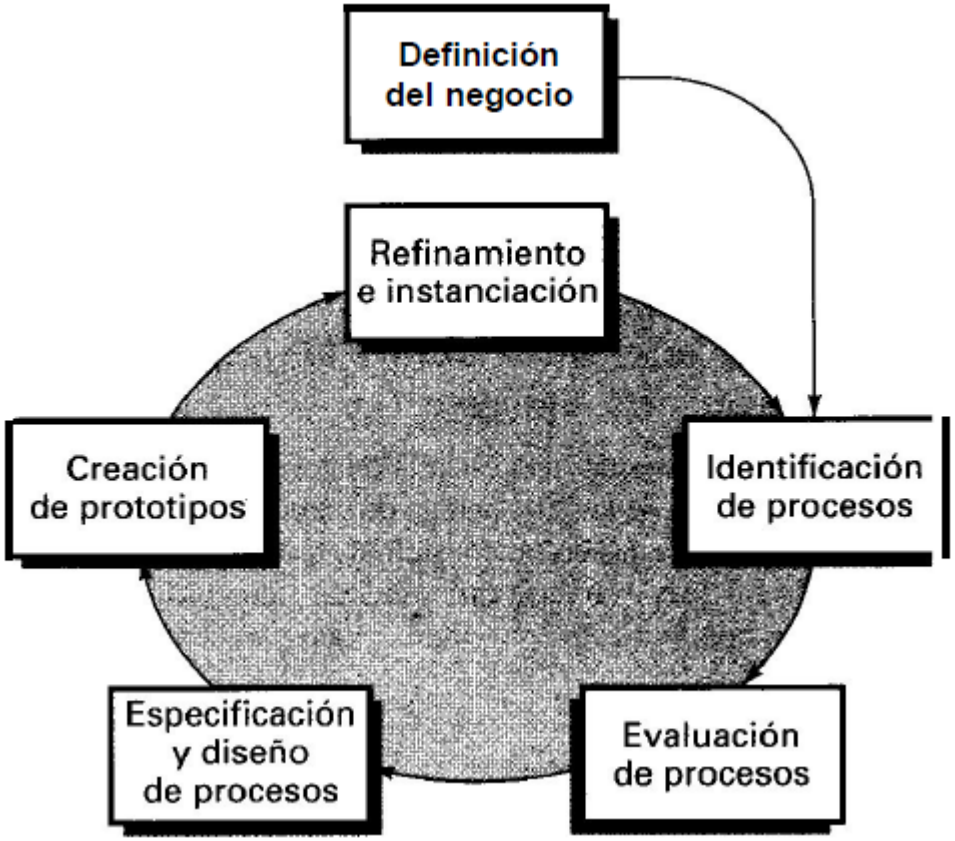
La Reingeniería de procesos de negocio, RPN (Business Process Reingineering, BPR2) va más allá del ámbito de las tecnologías de la información y de la ingeniería del software. Entre las muchas definiciones que se han sugerido para la RPN, se cuenta con una publicada en la revista Fortune [STE93]: «. . .la búsqueda e implementación de cambios radicales en el proceso de negocios para lograr un avance significativo».

Reingeniería de procesos de negocio

- **Procesos de negocios.**

Un *proceso de negocio es «un conjunto de tareas lógicamente* relacionadas que se llevan a cabo para obtener un determinado resultado de negocio» [DAV90].

Reingeniería de procesos de negocio

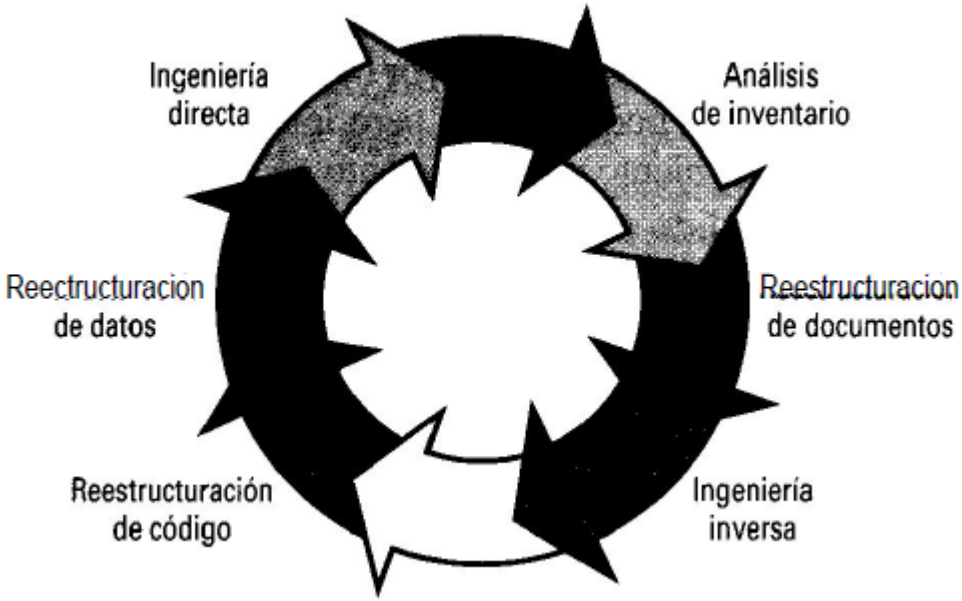


- Un modelo de Proceso de Negocio.

Reingeniería del Software

Este escenario resulta sumamente conocido: Una aplicación ha dado servicio y ha cubierto las necesidades del negocio de una compañía durante diez o quince años. A lo largo de este tiempo, ha sido corregida, adaptada y mejorada muchas veces. Las personas se dedicaban a esta tarea con la mejor de sus intenciones, pero las prácticas de ingeniería del software buenas siempre se echaban a un lado (por la urgencia de otros problemas). Ahora la aplicación se ha vuelto inestable. Sigue funcionando, pero cada vez que intenta efectuar un cambio se producen efectos colaterales graves e inesperados.

Un modelo de procesos de reingeniería del Software



Mantenimiento

En la actualidad tiene por término medio entre diez y quince años de antigüedad. **Aun cuando estos programas se crearon empleando** las mejores técnicas de diseño y codificación conocidas en su época (y la mayoría no lo fueron), se crearon cuando el tamaño de **los programas y el espacio de almacenamiento** eran las preocupaciones principales. **A continuación,** se trasladaron a las nuevas plataformas, se ajustaron para adecuarlos a cambios de máquina y de sistemas operativos y se mejoraron para satisfacer nuevas necesidades del usuario; y todo esto se hizo sin tener en cuenta la arquitectura global.

Mantenimiento

El resultado son unas estructuras muy mal diseñadas, una mala codificación, una lógica inadecuada, y una escasa documentación de los sistemas de software que ahora nos piden mantener.

Mantenimiento

El mantenimiento del software es algo que va mucho más allá de corregir errores. El mantenimiento se puede definir describiendo las cuatro actividades [SWA76] que se emprenden cuando se publica un programa para su utilización: *mantenimiento correctivo, mantenimiento adaptativo, mejoras o mantenimiento de perfeccionamiento y mantenimiento preventivo o reingeniería. Tan sólo el 20 por 100 de nuestros* esfuerzos de mantenimiento se invertirán «corrigiendo errores».

Mantenimiento

El 80 por 100 se dedicará a adaptar los sistemas existentes a los cambios de su entorno externo, a efectuar las mejoras solicitadas por los usuarios y a rehacer la ingeniería de las aplicaciones para su posterior utilización. Cuando se considera que el mantenimiento abarca todas estas actividades, es fácil ver por qué absorbe tanto esfuerzo.

Ingeniería Inversa

El *nivel de abstracción de un proceso de ingeniería* inversa y las herramientas que se utilizan para realizarlo aluden a la sofisticación de la información de diseño que se puede extraer del código fuente.

El nivel de abstracción ideal deberá ser lo más alto posible.

Ingeniería Inversa

Esto es, el proceso de ingeniería inversa deberá ser capaz de derivar:

sus representaciones de diseño de procedimientos (con un bajo nivel de abstracción);

y la información de las estructuras de datos y de programas (un nivel de abstracción ligeramente más elevado);

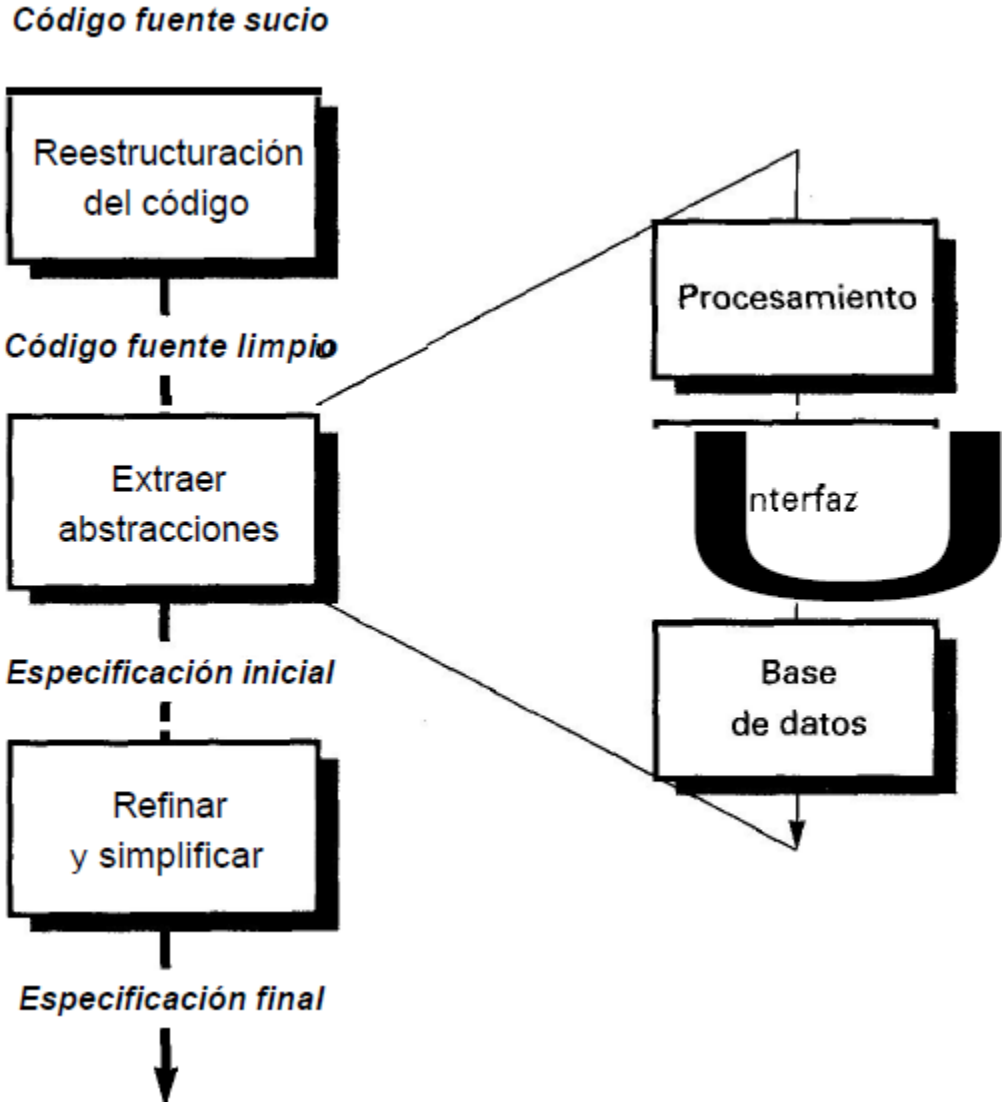
modelos de flujo de datos y de control (un nivel de abstracción relativamente alto);

y modelos de entidades y de relaciones (un elevado nivel de abstracción).

Ingeniería Inversa

A medida que crece el nivel de abstracción se proporciona al ingeniero del software información que le permitirá comprender más fácilmente estos programas.

Ingeniería Inversa



Ingeniería Inversa

La completitud mejora en proporción directa a la cantidad de análisis efectuado por la persona que está efectuando la ingeniería inversa. La *interactividud alude al* grado con el cual el ser humano se «integra» con las herramientas automatizadas para crear un proceso de ingeniería inversa efectivo.

Ingeniería Inversa

La *completitud de un proceso de ingeniería inversa* alude al nivel de detalle que se proporciona en un determinado nivel de abstracción. En la mayoría de los casos, la completitud decrece a medida que aumenta el nivel de abstracción. Por ejemplo, dado un listado del código fuente, es relativamente sencillo desarrolla una representación de diseño de procedimientos completa.

También se pueden derivar representaciones sencillas del flujo de datos, pero es mucho más difícil desarrollar un conjunto completo de diagramas de flujo de datos o un diagrama de transición de estados.

Ingeniería Inversa desde la Interfaz de usuario

Las IGUs sofisticadas se van volviendo de rigor para los productos basados en computadoras y para los sistemas de todo tipo. Por tanto el nuevo desarrollo de interfaces de usuario ha pasado a ser uno de los tipos más comunes de las actividades de reingeniería. Ahora bien, antes de que se pueda reconstruir una interfaz de usuario, deberá tener lugar una actividad de ingeniería inversa.

Ingeniería Inversa desde la Interfaz de usuario

Para comprender totalmente una interfaz de usuario ya existente (IU), es preciso especificar la estructura y comportamiento de la interfaz. Merlo y sus colaboradores [MER93] sugieren tres preguntas básicas a las cuales hay que responder cuando comienza la ingeniería inversa de la IU:

Ingeniería Inversa desde la Interfaz de usuario

¿Cuáles son las acciones básicas que deberá procesar la interfaz, por ejemplo, acciones de teclado y clics de ratón?

¿Cuál es la descripción compacta de la respuesta de comportamiento del sistema a estas acciones?

¿Qué queremos decir con «sustitución», o más exactamente,

qué concepto de equivalencia de interfaces es relevante en este caso?

Ingeniería Inversa desde la Interfaz de usuario

Gran parte de la información necesaria para crear u modelo de comportamiento se puede obtener mediante la observación de la manifestación externa de la interfaz existente. Ahora bien, es preciso extraer del código la información adicional necesaria para crear el modelo de comportamiento.

Es importante indicar que una IGU de sustitución puede que no refleje la interfaz antigua de forma exacta (de hecho, puede ser totalmente diferente).

Muchas Gracias

- Consultas.
- La próxima clase…..

Bibliografía

- ‘Ingeniería de software un enfoque práctico’, Roger S. Pressman, 6th ed. Editorial MC Graw Hill. 2005. Capitulo 30
- ‘Ingeniería del Software’, Ian Sommerville. 7th ed. Editorial Prentice Hall.
- Apuntes Cátedra Análisis de Sistemas II, Facena UNNE. http://exa.unne.edu.ar/informatica/anasistem2/public_html/home.htm